

Why we have to override hashCode method if equals method is overridden in Hash based classes HashSet and HashMap

Trainer's Name: **Ms. Shanmuga Priya**

Rule

- If two objects contents are equal according to equals(), they MUST return the same hashCode().
- If we override equals(), we should also override hashCode() so that object with the same content produce the same hash code.
- This is required for hash-based collections such as HashSet and HashMap to work correctly.

HashSet and HashMap use hashCode() and equals() for hashing operations.

- HashSet , HashMap -> overriding equals and hashCode
  removes duplicates
  improves search operation.

Index	Bucket
0	[ ]
1	[ ]
2	[ ]
3	[ ]
4	[ ]

HashSet and HashMap (Hash based classes) internally use a hash table, which is an array-based structure.

equals and hashcode method is overridden in the Employee class .

```
Employee e1 = new Employee(101, "John");
HashSet<Employee> empHashSet  = new HashSet<>();

empHashSet.add(e1);
```

e1.hashCode() → 200
bucket index calculation 200 % 5 = 0

So Java selects bucket with 0 index:

Index	Bucket
0	[e1]
1	[ ]
2	[ ]
3	[ ]
4	[ ]

```
Employee e2 = new Employee(102, "Peter");
empHashSet.add(e2);
```

e2.hashCode() → 202

Bucket index calculation = 202 % 5 = 2

Index	Bucket
0	[e1] ← hashCode = 200
1	[ ]
2	[e2] ← hashCode = 202
3	[ ]
4	[ ]

Employee e3 = new Employee(101, "John");
empHashSet.add(3)

e3.hashCode() → 200

200 % 5 = 0 --- bucket calculation

So Java selects bucket 0:

Index	Bucket	
0	[e1,e3]	--employee object with 200 hashcode will be stored here
1	[ ]	
2	[e2]	
3	[ ]	
4	[ ]	

Hash Collision

Employee e1 = new Employee(101, "John");
Employee e2 = new Employee(102, "Peter"); equals and hashcode method is overridden

e1.equals(e2) -> false

e1.hashCode() → 200 200 % 5 = 0 bucket index 0 [e1]

e2.hashCode() → 205 205 % 5 = 0 bucket index 0 [e1,e2]

Bucket index calculation :

e1 → Bucket 0
e2 → Bucket 0

Same bucket does NOT mean same object.

Bucket 0
↓
[e1, e2]

How searching operation is improved with equals and hashcode method? (With Hash based classes - HashSet and HashMap)

Employee e2 = new Employee(222,"John"); e2.hashcode() = 205 205 % 5 = 0 Bucket 0

Employee e3 = new Employee(111,"Abi"); e3.hashcode() = 200 200 % 5 = 0 Bucket 0

e2 and e3 in the same bucket, bucket with the index 0.

Suppose we want to search for e1.

Employee e1 = new Employee(111,"Abi"); - searching object

e1.hashCode() = 200

Java now directly goes to Bucket 0

Employee e1 = new Employee(111,”Abi”); - searching object

Employee e2 = new Employee(222,”John”); 205 % 5 = 0

Employee e3 = new Employee(111,”Abi”); 200 % 5 = 0

Bucket 0 contains:

Bucket 0 → [e2, e3](due to collision even though content are different, e2 and e3 in same bucket)

Java uses equals() to compare e1 with the objects in that bucket:
e1.equals(e2) → false
e1.equals(e3) → true

When equals() returns true, Java knows:

"This is the object I was looking for.“ e3 is returned

How duplicates are prevented with equals and hashcode method ?

Employee e1 = new Employee(101, "John");

HashSet<Employee> employees = new HashSet<>();
employees.add(e1);

e1.hashcode(); 200 - 200%5 – Bucket 0 [e1]

Employee e2 = new Employee(101, "John");
When employees.add(e2)
employees.add(e2)
↓
calls e2.hashCode()
↓
hash value = hashcode 200

hashCode = 200
Java calculates the bucket index

200
↓
bucket calculation 200 % 5 = 0
↓
Bucket 0

Control goes to Bucket 0
e1 is already there in bucket 1
Bucket 0 → [e1]
You are going to add e2
Java uses equals()
e1.equals(e2)

Result:
true

HashSet identifies e2 as a duplicate
e2 → NOT ADDED

HashSet.add(e2)

↓

e2.hashCode()

↓

calculate bucket index

↓

go to that bucket

↓

e1.equals(e2)

↓

true

↓

duplicate

↓

don't add e2

Content of the two object is same should have the same hashcode , so both objects will be in same buckets

But cotent with different objects will have different hashcode, due to hash collision, it will be in same bucket correct

THANK YOU